



대규모 트래픽 IT 기업의

AI Agent 실전 워크플로우

같은 인원으로 더 많이 시도하는 조직은
어떻게 일하는가

발표 • 45MIN

PART 1 - 6

2026.05.27

라이브 데모 • 15MIN

DEMO 1 - 5

오늘 한 문장만 가져가십시오

AI를 잘 쓰는 조직은
같은 인원으로
더 많이 시도한다.

Same headcount. More shots on goal.

오늘의 흐름

HOOK	오프닝 데모	2 min
PART 2	현업 IT 기업의 실전 패턴	8 min
PART 4	Claude · Codex · Gemini 비교	8 min
PART 6	계획과 실행의 분리	4 min

PART 1	AI Agent란 무엇인가	8 min
PART 3	직군별 에이전트 설계법	10 min
PART 5	Second Brain & 지식 관리	5 min
DEMO	라이브 데모 5가지	15 min

01

PART ONE · 8 MIN

AI Agent란 무엇인가

2022년이 변곡점이었다 / LLM과 Agent의 결정적 차이 / 맥락·문서·스킬
세 가지 축

2022년 이전, AI는 좁고 깊었다. 2022년 이후, 하나의 인터페이스가 됐다.



70년의 압축, 4년의 폭발



도구의 수가 **하나**로 줄었다.

번역 · 요약 · 코드 · 분석 · 설계 → ONE INTERFACE

LLM과 Agent는 무엇이 다른가

LLM

한 번의 응답

[입력] → [출력]

"시장조사 해줘" → 설명 텍스트.
대화를 계속 이어가야 결과가 나옵니다.

AGENT

반복 루프

[목표] → 계획 → 실행 → 확인

↑ _____ ↓

수정 → [완료]

"경쟁사 5곳 조사, 비교표, 전략 작성" → 완성된 문서 파일.
목표만 주면 끝까지.

한 줄로 외우십시오

LLM은 대답한다.
Agent는 끝낸다.

LLMs reply. Agents finish.

에이전트를 다루는 세 가지

CONTEXT01

맥락

에이전트 성능의 **80%**는 얼마나 좋은 맥락을 줬느냐에
서 결정됩니다.

→ 코드베이스 구조

→ 팀 코딩 규칙

→ 도메인 용어

AGENTS.MD02

팀 규칙 문서

한 번 써두면 **모든 세션에 자동 로드**. 우리 팀 규칙·금지
사항·스타일이 매번 살아납니다.

Project

- Python 3.11, pytest

- **print** 디버깅 금지

SKILLS03

워크플로 패키징

잘 작동한 프로세스를 묶어두면 다음엔 **한 줄**로 호출.
조직 자산이 됩니다.

\$ meeting-to-actions

\$ prd-to-spec

\$ competitor-analysis

FORMULA

|

Context + AGENTS.md + Skills = 반복 가능한 에이전트 시스템

맥락이 성능의 80%다

같은 모델, 같은 프롬프트라도 — 맥락의 깊이가 결과를 가릅니다.

- 우리 코드베이스의 폴더 구조
- 팀의 코딩 규칙과 금지사항
- 과거 의사결정의 이유
- 도메인 용어와 비즈니스 제약

80%

에이전트 성능 결정 요인

팀 규칙을 한 번만 쓰십시오

매번 같은 설명을 반복하지 않습니다. 한 파일에 우리 팀의 규칙·금지사항·스타일을 적어두면, 에이전트는 모든 세션 시작 시 이 문서를 자동으로 읽습니다.

→ 신입 온보딩 문서 = 에이전트 온보딩 문서

CLAUDE.md

프로젝트 개요

- Python 3.11, FastAPI, PostgreSQL
- 테스트는 pytest, 커버리지 80% 이상 유지
- 코드는 함수당 50줄 이하
- 타입 힌트 필수

금지 사항

- `print` 디버깅 금지 (logger 사용)
- 테스트 없이 PR 생성 금지

한 번 만든 워크플로는 조직 자산이 된다

STEP 1

잘 된 워크플로

발견

STEP 2

Skill로 패키징

규칙 + 템플릿

STEP 3

한 줄 호출

전체 프로세스 실행

회의록 → 액션 아이템, PRD → 개발 스펙, 인터뷰 → 인사이트.
반복되는 모든 것은 Skill이 될 수 있습니다.

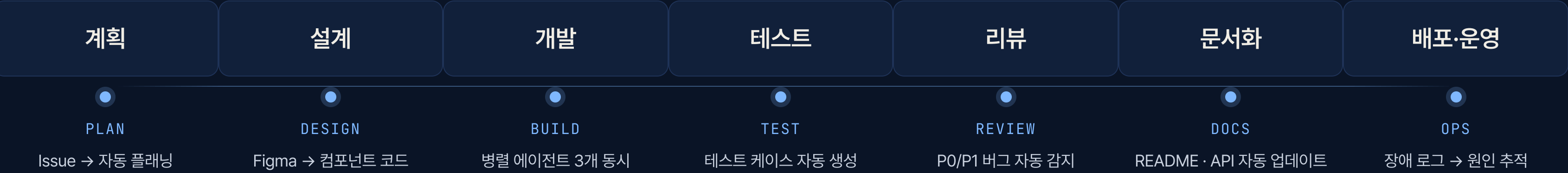
02

PART TWO · 8 MIN

현업 IT 기업의 실전 패턴

SDLC 전 단계에 AI가 붙는다 / Anthropic 내부 데이터 / 도입 성공과 실패의 분기점

전 단계에 에이전트가 붙는다



AI

반복 판단

반복되고, 검증 가능한 일

HUMAN

의사결정 · 승인

한 번 결정하면 되돌리기 어려운 일

어디에 무엇이 붙는가

단계	AI 활용 사례
계획	GitHub Issue → 자동 플래닝, PR 초안, 영향 범위 산출
설계	Figma 디자인 → 컴포넌트 코드 직접 생성
개발	병렬 에이전트 3개 동시 실행, N+1 쿼리 자동 수정
테스트	함수 작성 직후 테스트 케이스 자동 생성, 회귀 테스트
리뷰	P0/P1 버그 자동 감지, 보안 취약점 탐지
문서화	README · API 문서 · 마이그레이션 가이드 자동 업데이트
운영	장애 로그 → 원인 추적, PR Babysitter (CI/CD 자동 완료)

사람과 기계의 분업선

반복 판단은 **AI가,**
의사결정은 **사람이.**

Machines repeat. Humans decide.

1년 만에 무엇이 바뀌었나

Anthropic이 132명의 자사 직원을 대상으로 1년 전과 비교한 내부 조사 결과입니다. 체감이 아닌, 측정된 변화를 보십시오.

지 표	1년 전		현재
AI 사용 비율	28%		59%
체감 생산성 향상	20%		50%
PR 머지 수	기준		+67%
연속 자율 실행 (회/세션)	9.8	21.2 (+116%)	
인간 개입 횟수	6.2	4.1 (-33%)	

- 2배 증가

두 가지 지표 / 1 YEAR CHANGE

1년 만에 두 지표가 두 배

AI 사용 비율 / DAILY USE

28% → 59%

체감 생산성 향상 / PRODUCTIVITY

20% → 50%

27%

업무의 27%는 원래 하지 않았을 일.

AI가 없었다면 영원히 미뤄졌을 일들이 처리되고 있습니다.

- 테스트 코드
- 문서화
- 리팩토링
- 자잘한 버그 수정

잘 안 되는 회사는 이렇게 한다

"ChatGPT 써봐"

01

표준 없이 방치. 각자 알아서 쓰라는 말로 시작하면, 각자 알아서 안 씁니다.

개인기로만 쓴다

02

잘 쓰는 직원의 노하우가 다른 직원에게 흐르지 않으면, 조직 생산성은 제자리입니다.

회사 데이터와 단절

03

우리 자료를 못 보는 AI는 일반론만 답합니다. 도구의 80%가 맥락이라는 사실을 잊은 결과.

검수 기준 없음

04

"누가 책임지나?"가 정해지지 않으면 아무도 사용하지 않거나, 아무도 검토하지 않습니다.

잘 되는 회사는 이 순서로 한다

STEP 1	업무 선정	반복적이고 시간 많이 드는 것	예: 주간 보고서 · CS 정리 · QA
STEP 2	입력 정리	포맷 표준화 · 템플릿화	매번 다른 입력 → 매번 다른 결과
STEP 3	역할 부여	"너는 우리 회사 마케터야"	톤 · 맥락을 한 줄로 주입
STEP 4	형식 지정	표 · 불릿 · 보고서	결과물 구조를 미리 강제
STEP 5	검수 기준	이건 반드시 사람이 확인	책임선을 분명히 — 공백 차단
STEP 6	패키징	Skills로 반복 자동화	개인 노하우 → 조직 자산

표준화와 검수 기준 — 이 두 가지가 도입의 모든 것입니다.

03

PART THREE · 10 MIN

직군별 에이전트 설계법

개발자 · 기획자 · 디자이너 · 대표 — 같은 도구, 다른 사용법.

코드를 짜는 일에서 방향을 정하는 일로

BEFORE

코드 작성	50%
디버깅	30%
문서화·기타	20%

AFTER

방향 설정 · 검토	50%
AI 실행 감독	30%
의사결정 · 승인	20%

"코드를 짠다"는 비율은 줄어듦, "무엇을 짤지 정한다"가 늘어남.

다섯 가지 패턴이면 충분하다

01	CLAUDE.md PROJECT CONTEXT	프로젝트 맥락을 한 파일에 고정한다	100줄 → 모든 세션 자동 로드
02	Plan 모드 PLAN BEFORE EXECUTE	계획 → 사람 검토 → 실행	/plan → 승인 → 한 번에
03	병렬 에이전트 PARALLEL AGENTS	한 명이 동시에 세 명처럼 일한다	A: 로그인 · B: 가입 · C: 테스트
04	검증 조건 내포 COMPLETION CRITERIA	"완료"의 정의를 같이 줘라	pytest 통과 · 타입 에러 0
05	컨텍스트 관리 CONTEXT BUDGET	1M 토큰을 다 쓰지 마라 — 1/4 한계	250K 한계 + plan.md 캐시

프로젝트 시작 = 규칙 정의

● ● ● CLAUDE.md • 프로젝트 루트에 위치

프로젝트 개요

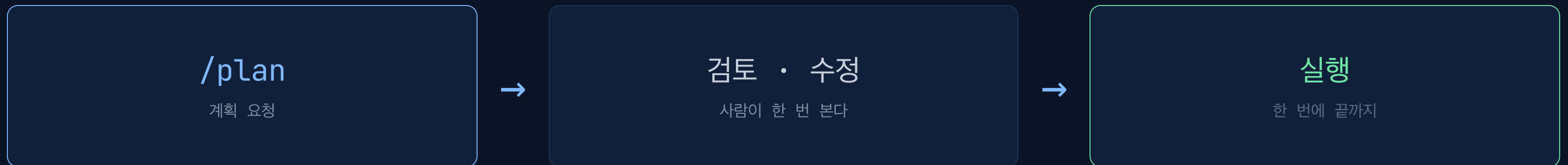
- Python 3.11, FastAPI, PostgreSQL
- 테스트는 pytest, 커버리지 80% 이상 유지
- 코드는 함수당 50줄 이하
- 타입 힌트 필수

금지 사항

- `print` 디버깅 금지 (logger 사용)
- 테스트 없이 PR 생성 금지
- `any/unknown` 타입 금지

세션마다 자동 로드. 100줄짜리 문서 하나가 매번 30분짜리 설명을 대신합니다.

계획 먼저, 코드는 나중에



이 순서가 핵심입니다. 바로 시키는 것이 가장 비싼 방법입니다.

한 명이 세 명처럼 일한다

AGENT A

로그인 기능

/auth/login · OAuth · JWT

■■■■■■■■■■ 78%

AGENT B

회원가입

/auth/signup · 검증

■■■■■■■■■■ 52%

AGENT C

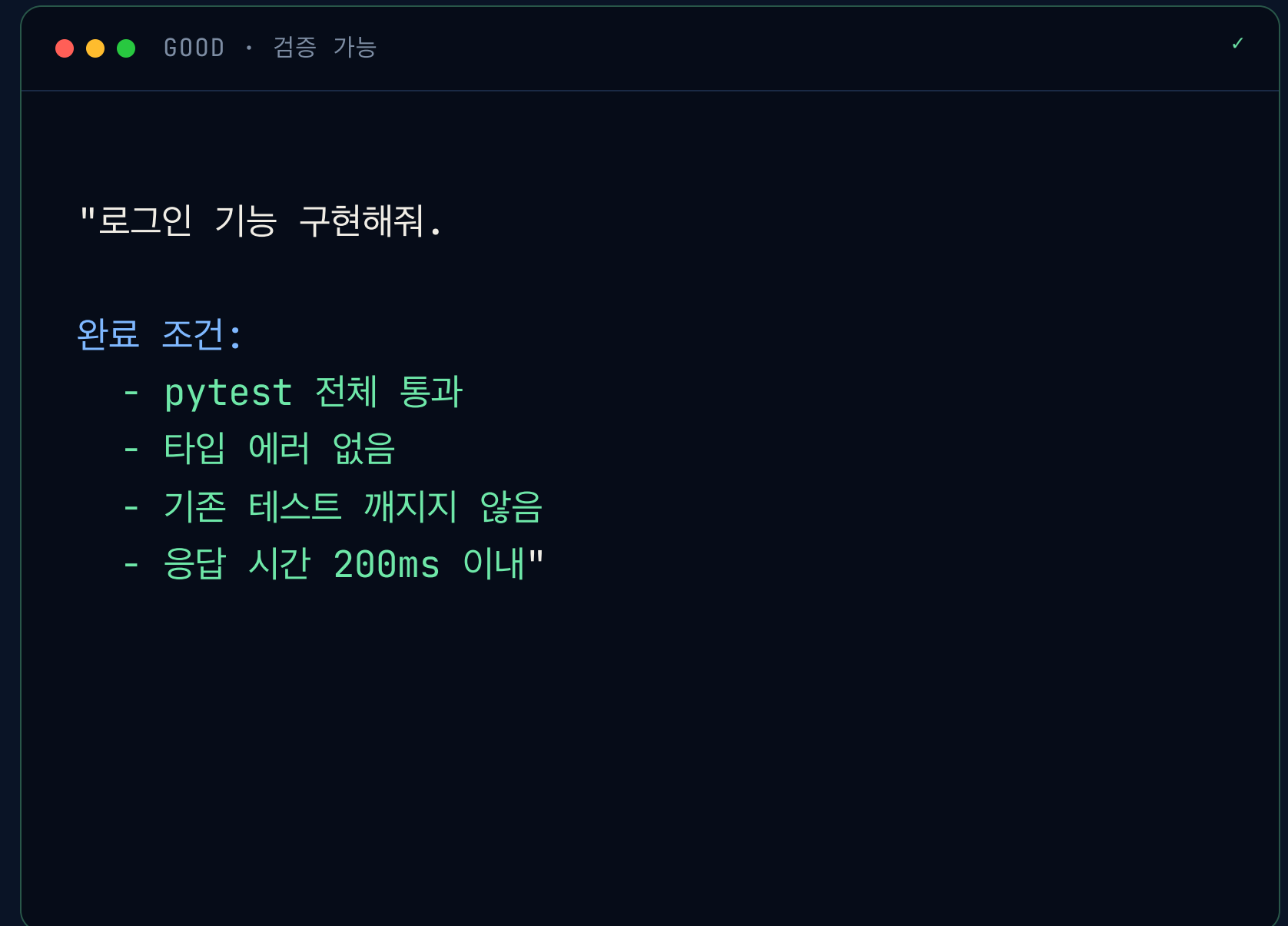
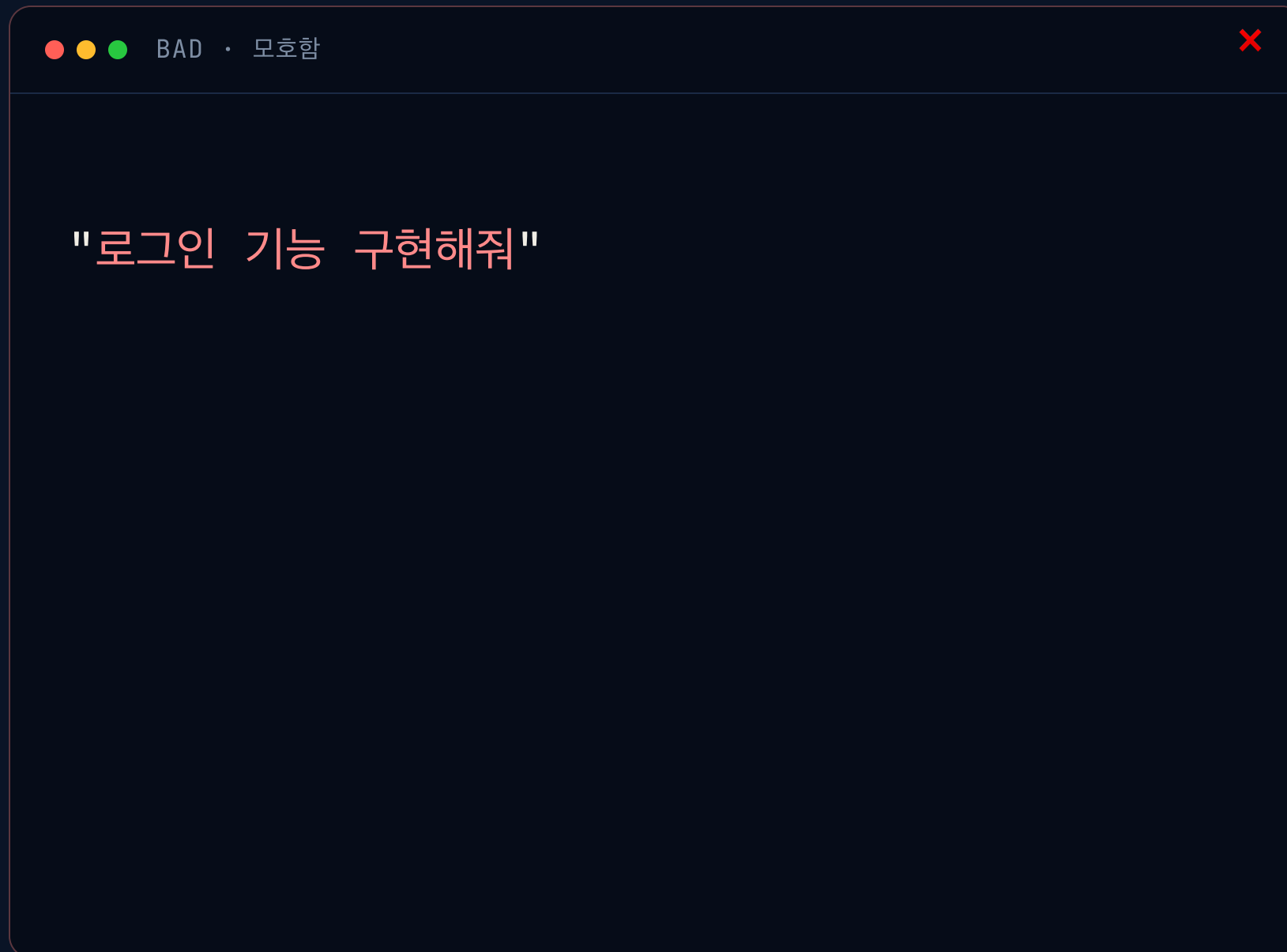
테스트 코드

pytest · 회귀 테스트

■■■■■■■■■■ 34%

세 터미널, 세 가지 작업, 하나의 사람. 컴퓨터 한 대로 팀이 됩니다.

"완료"의 정의를 같이 줘라



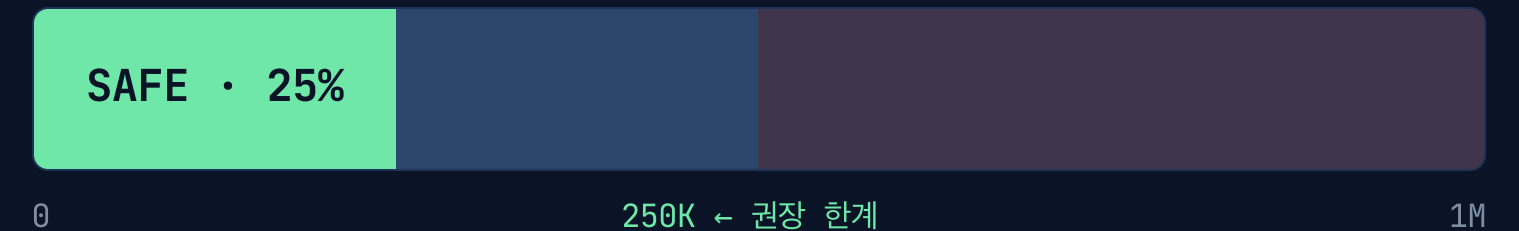
완료 조건이 명확하면, 에이전트는 스스로 검증하고 다시 시도합니다.

컨텍스트는 1/4 이하로

1M 토큰을 쓸 수 있다고 가득 채우지 마십시오. 컨텍스트가 길어질수록 에이전트의 집중력과 일관성은 떨어집니다.

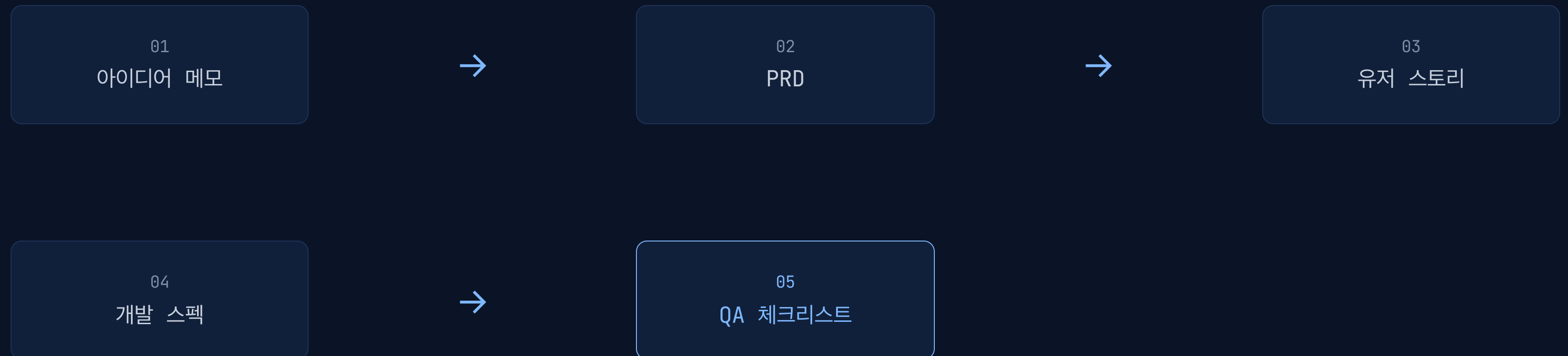
- 세션이 길어지면 새 세션 시작
- 핵심만 압축해서 전달
- plan.md를 살아있는 메모리로 사용

CONTEXT WINDOW • 1M TOKENS



컨텍스트는 예산입니다.
다 쓰면 다음 호출에서 가난해집니다.

아이디어에서 QA까지 같은 파트너로



각 단계의 결과물이 다음 단계의 입력. 한 명이 5단계를 모두 굴립니다.

경쟁사 분석을 한 번에

●●● Prompt · 경쟁사 비교표 생성

너는 SaaS 전문 PM이야.

아래 3개 서비스를 보고 비교표를 만들어줘.

컬럼:

- 핵심 기능
- 가격
- 타겟 고객
- 강점
- 약점

마지막에 우리가 틈새를 찾을 수 있는 영역을 제안해줘.

회의록 → 액션 아이템

●●● Prompt · 회의 정리

아래 회의록을 읽고:

1. 결정된 사항 (Owner, 기한 포함)
2. 논의만 하고 결정 못한 사항
3. 다음 회의 아젠다 제안

형태로 정리해줘.

Owner 없으면 "미정", 기한 없으면 "기한 없음"으로 표기

인터뷰 → 인사이트 추출

●●● Prompt · 고객 인터뷰 분석

고객 인터뷰 녹취록을 분석해줘.

1. 공통으로 언급된 불편함 TOP 3
2. 예상 못한 사용 패턴
3. 제품 개선 우선순위 제안

각 항목에 근거가 된 발언을 직접 인용해줘.

근거 없는 결론은 출력하지 말 것

3 - 3 . 디자이너

"AI가 디자인을 대신하는 게 아닙니다.
5개 검토할 시간에 20개를
검토하게 됩니다."

Not replacement. Bandwidth.

디자인은 사람, 변주는 AI

업무	AI 활용
UX 카피	버튼 텍스트 10가지 · 톤 변주
플로우 점검	엣지 케이스 자동 발견
Figma → 코드	Figma MCP → 컴포넌트 코드 생성
페르소나	고객 데이터 → 페르소나 카드
브랜드 가이드	카피 예시 → 톤앤매너 패턴 추출

FIGMA MCP 워크플로

- 01 Figma 링크 붙여넣기
- 02 디자인 컨텍스트 자동 읽기
- 03 기존 컴포넌트 재사용 코드 생성
- 04 Playwright로 브라우저 확인
- 05 불일치 자동 수정

우리 회사의 첫 번째 과제는 무엇인가

산업	추천 첫 번째 과제
제조업	불량 클레임 메일 + 검사표 → 원인 분석 보고서
유통업	판매 CSV → 다음 달 재고 / 프로모션 우선순위
서비스업	CS 로그 → 불만 유형 분류 + 개선 제안
부동산	매물 정보 → 투자 검토 보고서 초안
HR	JD 작성 · 면접 질문 생성 · 온보딩 문서

시작은 작게. 매일 하는 한 가지 보고서부터 자동화하십시오.

개인 → 팀 → 시스템

PHASE 1

1

개인 실험

팀원 1~2명이 특정 업무에 먼저 적용. 잘 되는 것· 안 되는 것을 빠르게 발견합니다.

PHASE 2

2

팀 표준화

잘 된 프롬프트를 팀 공용 템플릿으로. 한 명의 노하우가 전체로 흐릅니다.

PHASE 3

3

자동화

반복되는 프로세스를 **Skills / Automation**으로 패키징. 사람 손이 빠진 곳에 표준이 남습니다.

04

PART FOUR • 8 MIN

Claude · Codex · Gemini 도구의 결을 본다

하나만 쓰는 건 아깝다. 도구별 성격을 알고 단계별로 조합하는 것이 정답.

세 도구, 세 가지 달인

CLAUDE

읽고 분석하고
쓰는 것의 달인

Long-context reading.
Writing. Reasoning.

CODEx

코드베이스에서
장시간 혼자 일한다

Autonomous coding.
Long-running tasks.

GEMINI

Google 생태계와
연결된 것의 달인

Workspace native.
Multimodal.

강점과 주의점

항목	CLAUDE	CODEX	GEMINI
핵심 강점	긴 문서 이해 · 기획 · 글쓰기	대형 코드베이스 자율 실행	Workspace · 멀티모달
속도	빠름	느림 (3-4배)	중간
자율성	높지만 감독 필요	높고 지시 철저 준수	중간
컨텍스트	200K 토큰	25h 무종단 · 컴팩션	1M 토큰
생태계	MCP · Claude Code CLI	GitHub · Slack · Linear	Drive · Docs · Sheets
주의점	지시 가끔 무시	커뮤니케이션 딱딱함	코드 일관성 부족

시니어가 느낀 차이

CLAUDE

마감에 쫓기는 엔지니어 같은 느낌.
빠르고 인터랙티브하지만,
CLAUDE.md를 종종 무시하고
기존 파일에 함수를 무분별하게 추가하는 경향.

↩ FAST • INTERACTIVE • SLIGHTLY UNRULY

CODEX

5-6년차 시니어 같은 느낌.
느리지만 AGENTS.md를 절대 무시하지 않고,
자발적으로 리팩토링하고 가정을 재검토.
놔두고 커피 마시고 오면 끝나있음.

↩ SLOW • DISCIPLINED • WALK-AWAY-READY

언제 무엇을 쓰는가

업무	추천
보고서 · 전략 · 기획 · 복잡한 문서 분석	Claude
웹앱 개발 · 내부 도구 · 대형 코드베이스	Codex
Google Drive / Sheets · 이미지 · 영상 · 엔터프라이즈	Gemini
복잡한 PDF 여러 개 학습	NotebookLM
최강 조합	Claude(초안) → Codex(리뷰) → Gemini(연동)

세 도구를 어떻게 연결하는가

STEP 01

Claude

- 전략 · 보고서 초안
- 복잡한 문서 분석
- 요구사항 정리

STEP 02

Codex

- 코드 구현 · 리뷰
- 테스트 자동화
- PR 품질 검증

STEP 03

Gemini

- Google 생태계 연동
- 이미지 · 영상 처리
- 엔터프라이즈 배포

BONUS

NotebookLM

- 지식 축적 · 복습
- 팟캐스트 변환
- 학습 자료 정리

실전 파이프라인

Claude로 기획 초안 → Codex로 개발 구현 → Gemini로 Google Workspace 연동 → NotebookLM으로 지식화

05

PART FIVE • 5 MIN

Second Brain 지식 관리

하루에 쏟아지는 정보 중 뇌에 남는 건 5% 미만 — 두 번째 뇌가 필요한 시대.

5%

미만 / 뇌에 남는 비율

하루에 쏟아지는 정보 중
뇌에 남는 건 5%도 안 된다.

슬랙 수백 개 · 이메일 수십 통

뉴스레터 · 회의 5~6개

읽은 글 · 본 영상 · 들은 팟캐스트

두 번째 뇌가 필요한 이유

뇌는 아이디어를
보관하는 곳이 아니라,
연결하는 곳이다.

Brains connect. Storage belongs elsewhere.

하나로 다 하지 마라

01 • 수집

웹클리핑

북마크 · 회의록

02 • 저장

Obsidian

로컬 .md 파일

03 • 연결

백링크

Graph View

04 • 학습

NotebookLM

Gemini · 내 자료

05 • 생성

Claude

초안 작성

한 도구가 모든 걸 하면 모든 것을 못 합니다. 역할의 분리가 시스템을 만듭니다.

매일 한 줄씩 그래프가 자란다

01 아티클 읽기 · 자료 수집

02 Claude에게 인사이트 추출 요청

03 Obsidian에 노트로 저장

04 백링크로 관련 노트와 연결

● ● ● Prompt · Article → Note

"이 아티클을 분석해줘.

1. 핵심 인사이트 3개
2. 실무 적용 포인트
3. 연결 개념 태그

마크다운 노트 형식으로,
Obsidian에 붙여넣을 수 있게."

인터넷이 아니라, 내 자료만

학생

시험 정리

교재 PDF + 강의자료 + 논문 업로드 → "가장 자주 나오는 개념 정리해줘"

기획자

경쟁사 분석

경쟁사 보고서 10개 → "아직 안 된 포지션이 어디야?"

법무팀

계약서 검토

계약서 20개 → "불리한 조항 패턴 뽑아줘"

AUDIO OVERVIEW

업로드한 문서를 AI가 팟캐스트 형식으로 대화체 요약 → 출퇴근 중에 들었습니다.

• 5-5 • ROUTINE

5-5. 실전 루틴 / THE 10-MIN HABIT

하루 10분이면 시스템이 굴러간다

매일 • DAILY • 10 MIN

아침 5분 + 저녁 5분

• 06:30 • AM 정리

• 22:30 • PM 추출

어제 저장한 것 → Obsidian 태그 백링크

12:00

18:00

오늘 인상적인 것 → Claude로 인사이트

24:00

매주 • WEEKLY • FRIDAY

금요일 합성 1회

MON
5+5

TUE
5+5

WED
5+5

THU
5+5

FRI ★
+ 합성

NotebookLM에 이번 주 자료 모두 투입
→ "당장 써먹을 것 TOP 3"

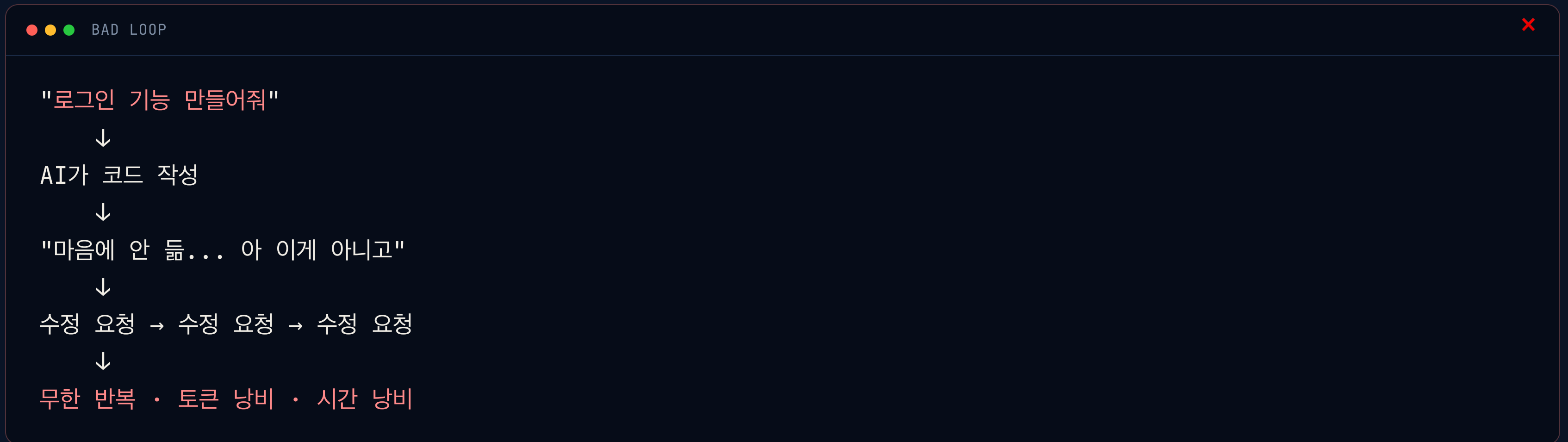
06

PART SIX • 4 MIN

Claude Code: 계획과 실행의 분리

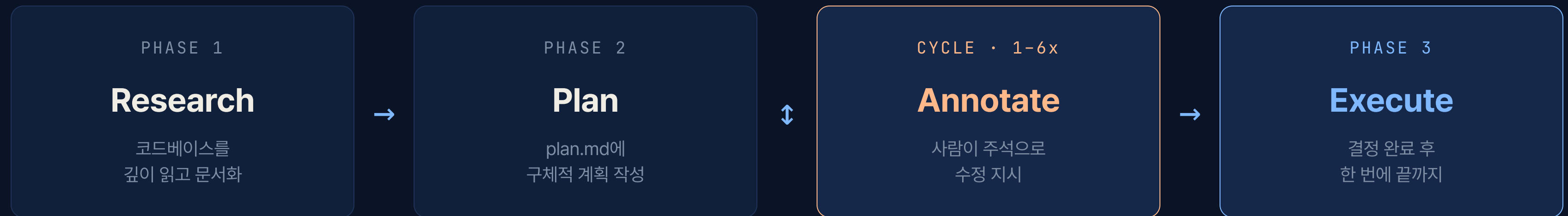
바로 시키지 마라. Research → Plan → Annotate → Execute. 9개월
의 결론.

"바로 시키기"가 가장 비싸다



문법 오류는 쉽습니다. AI는 보이지 않는 가정에서 실패합니다.

Research → Plan → Annotate → Execute



먼저 읽힌다 • research.md

● ● ● Prompt • 코드베이스 탐색

"우리 프로젝트의 인증 모듈을
deeply, with all intricacies로 읽고
research.md 파일에 정리해줘.

포함할 것:

- 현재 구조와 파일 경로
- 핵심 함수와 역할
- 의존성과 주의사항
- 변경 시 영향을 받는 영역"

코드를 짜기 전에 — 읽힌다. 모든 결정의 출발점.

계획부터 종이에 쓰게 한다

● ● ● Prompt · 계획 요청

"research.md를 기반으로
[기능] 구현 계획을
plan.md에 작성해줘.

포함:

- 수정할 파일 경로
- 코드 스니펫
- 트레이드오프
- 완료 조건

아직 구현하지 마. ← 이 문장이 핵심

● ● ● Annotation · 사람의 주석

3. 인증 토큰 처리

현재: JWT를 localStorage에 저장

수정: httpOnly 쿠키 사용,
refresh token 분리

"주식 반영해서 갱신해줘.

아직 구현하지 마."

→ 1~6회 반복

주식 반복 사이클 — 1~6회

- 1 plan.md 열기
AI가 생성한 구현 계획서 확인
- 2 인라인 주식 추가
내 판단과 다른 항목에 직접 코멘트
- 3 "갱신해줘, 아직 구현하지 마"
이 문장이 루프의 핵심 — 구현 금지 명령
- 4 만족할 때까지 반복
평균 1~6회 반복 후 Execute 진행

plan.md · 주식 예시

3. 인증 토큰 처리

현재: JWT를 localStorage에 저장

<!-- ❌ httpOnly 쿠키로 바꾸어야 함.
보안 정책 때문. -->

수정: httpOnly 쿠키 사용,
refresh token 분리

4. API 응답 캐싱

Redis TTL: 5분

<!-- ⚠ TTL을 30초로 줄일 것.
실시간 가격 데이터라서. -->

→ "주식 반영해서 갱신해줘.
아직 구현하지 마."

이제, 한 번에 끝까지

●●● Prompt · 실행

"plan.md의 모든 태스크를
완료할 때까지 멈추지 마.

규칙:

- 불필요한 주석 추가 금지
- any / unknown 타입 금지
- 변경할 때마다 타입 체크 실행
- 완료한 태스크는 plan.md에서 체크 표시"

앞 두 단계에서 결정이 끝났기 때문에, 실행은 빠르고 일관됩니다.

같은 3시간, 다른 결과

일반 방식	
시간	3h
토큰	많음
결과	수정 반복

계획 / 실행 분리	
시간	3h
토큰	적음
결과	한 번에 올바름

보너스: plan.md는 팀 공유 문서이자, 다음 세션의 맥락 캐시가 됩니다.

오늘 한 장만 남긴다면

01 Agent는 목표를 받고 끝까지 실행한다.
→ 지시보다 "위임하는 방법"을 배워라.

02 Claude · Codex · Gemini는 각자 다른 달인이다.
→ 업무 단계별로 조합하는 것이 정답.

03 AI를 잘 쓰는 조직은 같은 인원으로 더 많이 시도한다.
→ 도입의 핵심은 표준화와 검수 기준.

월요일 아침, 이 셋 중 하나

● EASY · 10 MIN

NotebookLM 첫 질문

지금 보는 문서 하나 올리고 질문해보기. 오늘 안에 끝납니다.

● MEDIUM · 30 MIN

CLAUDE.md 작성

나 / 팀이 AI에게 알려줄 규칙 100줄. 다음 주가 다릅니다.

● HARD · 2 HOURS

Skill 패키징

반복 업무 하나를 Skill로. 조직 자산이 시작됩니다.



LIVE DEMO • 15 MIN

화면을 캡니다.
지금부터는 라이브.

5개 데모, 각 3분. 네트워크가 흔들리면 녹화본으로 전환합니다.

PRD 한 장 → 작동하는 대시보드

PRD • 팀 주간 목표 대시보드

- 배경 팀원 5명 주간 목표를 한눈에 보는 도구 없음
- 목표 이름 + 주간 목표 3가지 입력 → 카드 표시
- 기술 단일 HTML • localStorage • Tailwind CDN
- 완료 입력 / 표시 / 체크박스 / 새로고침 유지 / 빈값 경고

CUE CARD

"먼저 plan.md로 보여주고, 승인하면 구현 시작" — 두 번 강조하기.

Claude Code • 전달 프롬프트

위 PRD를 읽고
팀 주간 목표 대시보드를 만들어줘.

먼저 구현 계획을 plan.md로 작성하고
나에게 보여줘.

승인하면 그때 구현 시작해.

Skill을 직접 만들어 본다

만들 SKILL

meeting-to-actions

회의록 → 결정사항 / 미결사항 / 다음 아젠다로 정리하는 Skill.

CUE CARD

생성 직후 실제 회의록 한 장으로 바로 테스트해 보여주기.

\$ skill-creator

\$ skill-creator

이름: meeting-to-actions

목적: 회의록을 결정 / 미결 / 다음
아젠다 형식으로 정리

규칙:

- Owner 없으면 "미정"
- 기한 없으면 "기한 없음"
- 근거 인용 포함

→ skill.md 생성 완료

브라우저에서 바로 UX 분석

경쟁사 사이트를 띄워둔 채로, 확장 프로그램에서 바로 UX 진단을 받습니다.

→ CTA 버튼 명확성

→ 정보 구조의 우선순위

→ 사용자가 먼저 봐야 할 것

CUE CARD

사이트는 청중에게 익숙한 것 — 미리 두 개 준비.

Browser Extension · UX Audit

이 페이지의 UX 문제점을 찾아줘.

평가할 것:

- CTA 버튼 명확성
- 정보 구조
- 사용자가 먼저 봐야 할 것

각 문제마다 화면 좌표와
구체적 개선안을 함께.

Obsidian Vault, 살아있는 두 번째 뇌

STEP 1

Obsidian 폴더 구조 보여주기

STEP 2

Claude로 오늘 아티클 → 노트 변환

STEP 3

Graph View로 연결망 시각화

[OBSIDIAN GRAPH VIEW]

실제 화면을 여기서 공유

슬라이드와 문서를 자동으로

A • PPT

자동 슬라이드

"아래 내용으로 발표 슬라이드
5장을 만들어줘 (.pptx)"

실제로 이 슬라이드도 같은 방식으로 시작했습니다.

B • AUDIO

NotebookLM 오디오

미리 준비된 세션에서 **Audio Overview** 재생.
AI가 두 명의 화자처럼 토론하듯 요약합니다.

출퇴근 시간이 학습 시간이 됩니다.

한 번 더, 마지막으로

같은 인원으로
더 많이 시도하십시오.

THANK YOU • Q&A